

Social Choice and Welfare

A Clustering Algorithm for Proportional Representation

--Manuscript Draft--

Manuscript Number:	
Full Title:	A Clustering Algorithm for Proportional Representation
Article Type:	Original Paper
Corresponding Author:	Richard Henry Burkhart, Ph.D. Democracy Works Seattle, WA UNITED STATES OF AMERICA
Corresponding Author Secondary Information:	
Corresponding Author's Institution:	Democracy Works
Corresponding Author's Secondary Institution:	
First Author:	Richard Henry Burkhart, Ph.D.
First Author Secondary Information:	
Order of Authors:	Richard Henry Burkhart, Ph.D.
Order of Authors Secondary Information:	
Funding Information:	
Abstract:	We describe a new kind of voting algorithm, PR clustering, for proportional representation, an optimized clustering algorithm, when voters rank or rate the candidates in a jurisdiction. Our method converts these preferences to fractional points to use techniques from modern scientific computing, where the clusters, or overlapping voting blocks, are identified by vectors that summarize how voters in the clusters rank or rate their preferred candidates. Our results are compared with the traditional method, Single Transferable Vote (STV). Key techniques derive from discrete, continuous, and combinatorial optimization, making issues of complexity, non-linearity, and sensitivity more transparent. Our test files are full ballot sets from 40 elections, mostly from Scotland. Conclusion: STV is generally good but certainly not optimal. It duplicated our optimized PR clustering set for 70% of the districts and did well on most of the rest, but it hides other good electoral outcomes. An improved version, which we call DTV, or Distributed Transferable Vote performed better than STV, with 85% exact matching to PR clustering. Even Borda voting did as well as STV for exact matches, though not as well on the mismatches, as measured by our objective function.

A Clustering Algorithm for Proportional Representation

Dr. Richard H. Burkhardt, Democracy Works, Seattle, WA

(dick_burkhardt@centurylink.net, orcid: 0009-0002-1111-1964)

Abstract. We describe a new kind of voting algorithm, *PR clustering*, for proportional representation, an optimized clustering algorithm, when voters rank or rate the candidates in a jurisdiction. Our method converts these preferences to fractional points to use techniques from modern scientific computing, where the clusters, or overlapping voting blocks, are identified by vectors that summarize how voters in the clusters rank or rate their preferred candidates. Our results are compared with the traditional method, *Single Transferable Vote* (STV). Key techniques derive from discrete, continuous, and combinatorial optimization, making issues of complexity, non-linearity, and sensitivity more transparent. Our test files are full ballot sets from 40 elections, mostly from Scotland. Conclusion: STV is generally good but certainly not optimal. It duplicated our optimized *PR clustering* set for 70% of the districts and did well on most of the rest, but it hides other good electoral outcomes. An improved version, which we call DTV, or *Distributed Transferable Vote* performed better than STV, with 85% exact matching to *PR clustering*. Even Borda voting did as well as STV for exact matches, though not as well on the mismatches, as measured by our objective function.

Key Words: proportional representation, ranked choice voting, single transferable vote, clustering algorithms, fuzzy sets, k-means, discrete optimization, continuous optimization, combinatorial algorithms, non-linear complexity, scientific computing.

1 Introduction

Proportional representation is based on the proposition that non-dominant groups or minorities should be given direct representation in legislative bodies in proportion to their size. Thus to determine the balance of power,

voters often cast their ballots, not for individuals, but for political parties in European-style parliaments. In this *party list* system the number of parliamentarians representing a party is in proportion to the vote received by that party, with the actual individuals determined by the party's priority list of candidates. In addition to political party, our clustering algorithm for proportional representation could be based on a wide variety of group identities or interests, such as cultural or ethnic group, geographic location, socio-economic class, labor, business, environment, urban, suburban, rural, or other interest.

Such groupings are possible when voters rank or rate candidates even when voting blocks are ambiguous or overlapping. Our concept is that voting blocks may be identified by the algorithm as clusters of voters who tend to rank or rate the candidates in similar ways, or at least their top candidates. Note, however, that such a system could be combined with an option for simply choosing the party or group, an option whose simplicity would appeal to many voters, in addition to the parties themselves. In this case, the party or group would be authorized to do the ranking or rating for the voter. Although we use the language of candidates and voters, the context could be one of surveying a group on alternatives or options, or when estimated rankings or ratings could be derived from other data. Our algorithm is coded in Fortran 2003 with code and data available.

The standard algorithm for proportional representation for ranked ballots is *Single Transferable Vote* (STV, see Wikipedia, for example) which adds a quota system to the elimination method called *Instant Runoff Voting* (IRV) in the US, now promoted as *Ranked Choice Voting*. A prime motivation for our study was to test the actual proportionality of STV. It just wasn't clear, though STV implicitly identifies as a cluster all those voters who top rank a particular candidate either directly or during the course of a candidate election and elimination procedure. Yet, as clusters, these *candidate clusters* are quite crude: They end up being the size specified by the quota and they partition the ballots. Real world voting blocks are of varying sizes, may represent multiple candidates, and have overlap and complexity, whether or not they can be identified with particular political parties, interest groups, or alliances.

Proportionality means that the strength by which a voting block is represented by the set of elected candidates should match the size of the voting block as closely as possible, balanced over all significant voting blocks. This is exactly what our algorithm does, using combinatorial methods and suit-

able mathematical objective functions. We have also extended STV itself in a straightforward way to produce more realistic clusters (section 6.2.1) in addition to modifying STV to a *Distributed Transferable Vote* (DTV). STV transfers the weight of a candidate ranked by a particular ballot to the next ranked candidate that remains *hopeful*, i.e., not elected or eliminated, either in whole, if the candidate is to be eliminated, or in part, if it is to be elected. DTV distributes this same weight, but in decreasing fractions over the subsequent hopefuls. Thus DTV incorporates the greater realism of point based methods, such as Borda, to compensate for the over emphasis on top ranking by STV. Our result is an improvement from a 70% match to our PR clustering set to a 85% match, suggesting an upgrade of STV to DTV in computerized systems. Other variations of STV, such as the Warren method [9] were virtually the same in practice, but we use it with our DTV method since it often matched *PR clustering* for one more district.

However our extended STV clusters still fall short, as the assignment part of our algorithm (section 7) may show that a different set of candidates may be a better fit to the extended STV clusters than the original STV electeds. Besides STV it has been instructive to compare our optimized clustering to other common methods that may incidentally yield partial proportionality, such as IRV, the *Top Vote*, *Borda*, *Condorcet*, and *Approval*. A robust tournament method (section 9.4), based on combinatorial comparisons of possible elected sets, yielded reasonable but not improved results.

In Phase I of the algorithm (*Ballot Consolidation*, section 3) starts with our first version (*strong ranking*) where we partition the original ballots into sets which have the same top ranked candidates (slates of candidates), which we call *slate ballots*. In our second version (*weak ranking*), some candidates may be ranked equally. In our third version (*rating*), some candidates may receive a positive rating while others may get a negative rating, with partitioning based on both top rated and bottom rated candidates. These slate ballots are then represented as primitive *slate clusters*, characterized by the number of original ballots represented (the size or weight of the cluster) and by the mean rating vector (the average point value for each candidate given a specified conversion of ranking or rating levels to points, see *Voting*, section 2). This constitutes the discretization phase of scientific computing.

In Phase II (*Cluster Analysis*, sections 4 through 6) we *converge sets of regular clusters*, whose members are slate clusters. The regular clusters in a set may be partially overlapping, and some slate clusters may partially or fully represent independent voters (the pseudo-cluster of *independents*). That

is, for each slate cluster, its sum of memberships over the regular clusters may be less than 1, with the remaining membership belonging to the residual voting block of independents. This makes these cluster sets *fuzzy sets* in the sense of Zadeh [1] - with partial, real number memberships. Thus our method is a variation on the traditional fuzzy k-means algorithm [5].

We use a nonlinear global optimization procedure, with convergence from multiple *initial cluster sets*. The measure of similarity between slate clusters and a converging regular cluster, section 4, is derived from correlations between their mean rating vectors. Our careful convergence process, section 6.1, includes the merging of well correlated clusters using a maximal clique algorithm [4], p 109-114, deletion of very small clusters, and a damped Newton method with line search [3] when needed. The initial cluster sets, section 6.2, come from several simpler methods of clustering, including clustering coefficients [6] and our extended STV, plus 15 randomized sets of overlapping clusters which top rank slates of 1, 2, or 3 candidates.

The convergence process generally yields several distinct converged cluster sets, often with varying # clusters per set. We evaluate these by a clustering objective function, section 5, which rewards sets which are more encompassing and better defined. In particular, we penalize small cluster sizes and significant cluster correlations which fall short of the deletion / merging criteria, also penalties for large sets of independents. What is maximized is the sum over the regular clusters of their size times weak measures of their densities, to be maximized, and of their boundaries, to be minimized. This we call the *coherent vote*, signifying the higher value placed on larger clusters which are more compact and less diffuse.

In Phase III (*Candidate Matching*, section 7), for a given set of converged clusters, we match candidates to the clusters which tend to rate those candidates highly, while penalizing significant deviations from proportionality. The proportionality takes into account the fact that candidates may be partially matched to more than one regular cluster or to the independents. Partial matching is represented by a fraction that depends on the product of the mean rating of the candidate by the cluster and the size of the cluster, section 7.1. That is, the higher the mean vector of a voting block rates a candidate, and the bigger the block, the more of that candidate's vote, or *portion* of the legislative vote, should represent that block. We define this *portion* as the product of the mean rating of the candidate by the voting block times the size of the voting block, normalized so that the sum of the candidates *portions* over all the voting blocks is 1 (47), with portions zeroed

out if the mean rating is small, or “in the noise”. This is the foundational concept of *PR clustering*.

Sometimes a candidate who is weaker overall (lower average rating) will get elected by providing effective representation to more than one voting block (better proportionality). Here the independents count as a defacto voting block and may elect an independent candidate when the regular clusters are deficient. In this context, an elected set of candidates represents the voting blocks proportionately if the sum of the portions from those candidates for each voting block approximates the size of that voting block in units of elected positions.

A branch-and-bound algorithm [4], p 141-3, described in section 7.2, evaluates the potential sets of electeds using an objective function which penalizes large deviations from proportionality while maximizing the weighted average over the clusters of the mean ratings of the elected candidates. This *matching objective* multiplies the clustering objective to get a *full objective* function, section 8. Maximizing this full objective determines the final set of electeds. We note that occasionally the second best cluster set may be almost as good as the best one but yield a better matching, hence a higher full objective.

By way of example, Ireland and Scotland implement proportional representation using STV by first forming super districts with 3 or 4 electoral seats, then having voters rank the candidates, who run at large rather than for individual seats. This is in a context where a variety of political parties mobilize different constituencies, though sometimes independents run as well, and voters may cross party lines. We obtained 33 complete sets of digital ballots from Scotland (Glasgow and Scottish-Borders), 2 from Ireland (Dublin), and 5 from the US, constituting the 40 data sets used for our testing, section 9. Of the 40 districts, 3 elected two candidates, 17 elected three candidates, 18 four candidates, and 2 five candidates.

Our rating method uses a version of what is sometimes called score or range voting, or cardinal versus ordinal voting (= ranking). The difference is that we impose an automatic *average restriction* on ratings or weak rankings to offset tactical voting. 0 is interpreted as a neutral point value for ratings, and a voter’s point values for the rated candidates should average close to 0, or we adjust the rating levels to do this. Similarly, for weak ranking the point values for the ranked candidates should sum to the corresponding sum for strong ranking, at least approximately, or we adjust the ranking levels to make this happen.

Clustering reveals many of the complexities of voting that are hidden by

traditional algorithms like STV. You can easily see when a particular election gives a very stable result or when it is sensitive to small changes in the voting or parameters. For scholars this brings seemingly endless opportunities for more insightful analysis and algorithmic variations. For voters it is another matter altogether. They would like the impossible (Arrow’s theorem [8]) - a simple, cut and dried method that is clearly superior to all others, with no ambiguities or paradoxes.

Nevertheless, both candidates and citizens have learned to live with the luck of the ballot in close elections, even deciding ties by flips of a coin. Citizens need to learn that methods of voting themselves have a similar character. Instead of wishing for perfection, or giving in to cynical resignation to imperfection, citizens need to establish reasonable statistical performance goals to be realized over many election cycles. Simulations can be an excellent starting point, but are no substitute for actual experience. The kind of clustering analysis and candidate fitting presented here can give insight into particular elections, but its real promise is to provide better performance in practice according to criteria developed and monitored through democratic means.

The great danger of course is that powerful factions can find ways to corrupt even the best democratic processes, or political parties may find ways to eliminate proportional representation, as they have in the past in the U.S., if the process is not carefully constructed and maintained in bipartisan ways.

2 Voting

Let’s look in more detail at the methods of voting accommodated by our code. First, we provide options for the essential task of translating rankings or rating levels into point values, hence vectors of real numbers that may be clustered. This is in the context of truncated ranking, either to accommodate limitations on computer memory, to intentionally restrict the burden on voters, or to discourage tactical voting like ranking prominent opponents last regardless of qualifications. That is, the user may specify the maximum number m_r of candidate rankings to be used out of n_c candidates. Most often we have used 5 or 6.

The most obvious way to translate rankings into ballot points is via the standard Borda Count: the i^{th} top ranked candidate is assigned $p_t(i) = m_r + 1 - i$ points, which has an optimality property if the ranking is your only

information. However, voters often focus primarily on a few top candidates and want to give them a higher score than the Borda Count does. This desire can be accommodated in a variety of ways. For example, we could increase the Borda point increment d by a value Δ at each rank: Set $d = 1$ and $p_t(m_r) = 1$, then $p_t(i) = p_t(i + 1) + d$, for $i = m_r - 1, \dots, 1$ with $d = d + \Delta$ after each assignment. The *modified Borda* that we used throughout is for $\Delta = 0.5$ with top point normalization so that $p_t(1) = 10.0$. For $m_r = 5$ the result is (10.0, 7.0, 4.5, 2.5, 1.0) instead of the standard Borda of (10.0, 8.0, 6.0, 4.0, 2.0). Or you could try a geometric progression. Our clustering results are quite sensitive to larger changes in Δ , such as dropping it to 0.2, as documented in section 9.3, since this point-conversion parameter is at the very base of the algorithm.

Another option is to use a rating method and assign ballot points directly. An example would be $m_t = 6$ rating levels, such as (1,...,6) = (outstanding, very good, good, not-so-good, deficient, unacceptable). Reasonable point values might be $p_t = (10.0, 6.0, 2.0, -2.0, -6.0, -10.0)$, with unrated candidates getting the neutral value of 0. The problem is that ratings are subject to extremes of tactical voting, such as rating one group or individual at the top and everyone else at the bottom, which is actually a kind of approval voting. To discourage this and to make individual voter's ratings more comparable, we impose an *average restriction*. That is, we automatically adjust the rating levels for each voter by an iterative process so that the average point value of the rated candidates is close to 0. With each sweep through the candidates, we identify the most suitable candidate for a rating level increment or decrement. If only one candidate is rated we select another candidate at random and assign it the rating of opposite sign that makes the average closest to 0.

We use the same method for weak ranking, which is like the strong ranking standard except that some candidates may be ranked equally. In this case the neutral ranking level is not 0 but the average ranking of $m_r(m_r + 1)/2$, so that ranking levels are adjusted as needed to approximate this value.

Another issue is how to get voters to rank or rate a sufficient number of candidates, not just their favorite. For the *modified Borda Count* of Peter Emerson ([2]), a voter who ranks $n < m_r$ candidates gets only $n + 1 - i$ points for his or her i^{th} top candidate instead of $m_r + 1 - i$, with all unranked candidates getting 0 points. At the opposite extreme of the *full vote*, every voter gets exactly the same point total, any shortfall spread equally as a fraction among the unranked candidates. Thus for $n < m_r$ ranked candidates, the

missing Borda point total is $u(n) = m(m + 1)/2$ where $m = m_r - n$, so that each unranked candidate gets $p_u(n) = u(n)/(nc - n)$ points. We also provide for an option, the *partial vote*, where p_u may be scaled by a fraction f_u , such as, $f_u = 0.5$, with $f_u = 0$ more like Emerson’s method. Another option is to down-weight ballots which rank fewer than a specified number of candidates p_r . For $p_r = 3$, a ballot which ranks $n_r = 1$ candidates could be counted as $1/3$ of a vote, with a $n_r = 2$ ballot counted as $2/3$ of a vote, for example.

Another issue is that lower rankings often come with not just fewer points but more uncertainty, especially p_u , so how should this affect the averaging of the ballot rating vectors during ballot consolidation? This concern is addressed by interpreting the point values as random variables with specified variance, as described in section 3.1.

3 Ballot Consolidation

Our discretization consists of two stages of ballot consolidation. The first stage is a pre-processing consolidation to *initial ballots*, with more detail in section 3.1. This includes both removing marginal candidates and restricting the number of ranked or rated candidates used from each ballot, then combining ballots which the duplicate ballots by simply assigning their vote weight to be the number of duplicates. Again, this could yield a big reduction in computational and memory load, a reduction in the burden on voters, a reduction in tactical voing, and a stronger incentive for groups to form political alliances centered around promoting compatible candidates rather than mudslinging against the opposition. The rating option is provided for a situation with a goal of including both strongly liked and disliked options, such as certain surveys.

In the second stage, initial ballots which top rank the same small slate of candidates in a similar way are consolidated into *slate ballots*, which normally meet a minimum weight requirement, see section 3.2. These slate ballots, which continue to form a partition, may be regarded as *slate clusters*, characterized by their weights and mean point vectors. The slate clusters then become the members of the converged *clusters* which constitute our voting blocks, section 4.

3.1 Initial Ballots

We identify marginal candidates by a suitable point cutoff, using our modified Borda (section 2). This may be computed when reading in the ballots without storing them. When combined with the cutoff m_r on the number of rankings to be used, the second reading of the ballots often yields a major consolidation of the ballots and a big drop in required computer memory.

To further consolidate ballots with rankings of variable certainty, we compute the maximum likelihood estimate of the mean, rather than the standard averaging used in the Borda Count. That is, each initial ballot b is modeled as a random vector whose mean is its ranking point vector μ_b , with $0 \leq |\mu_b| \leq 10$, and a reasonable specification for its variance vector $v_b = \sigma_b^2$ which increases as μ_b decreases.

Let σ_1 be the standard deviation for the top ranked candidate and σ_u be the larger standard deviation for the unranked candidates, with multiplier $m_u = \sigma_u/\sigma_1 > 1$. Then for ranking levels in between we square the linear function from $\sqrt{\sigma_1}$ to $\sqrt{\sigma_u}$, to get a quadratic transition that rises rapidly as the unranked level is approached. However, the actual sigmas we want are for the point values, not the integer ranking levels. This means scaling the ranking level sigma by the approximate spread around its point value, as indicated by the gaps between the point values. Suitable values are $\sigma_1 = 0.35$ and $\sigma_u = 1.40$ with multiplier $m_u = 4.00$.

When $n_r < m_r$ candidates are ranked, then we can model each unranked candidate as a mixture of random variables $U(n_r) = [(X_{n_r+1} + \dots + X_{m_r}) + X_u + \dots + X_u]/(n_c - n_r)$ with mean unranked point value $u(n_r)$ and variance $v_u(n_r)$

$$\begin{aligned} u(n_r) &= \left[\sum_{i=n_r+1}^{m_r} p_t(i) + (n_c - m_r)p_u \right] / (n_c - n_r) \\ v_u(n_r) &= \left[\sum_{i=n_r+1}^{m_r} v_t(i) + (n_c - m_r)v_u \right] / (n_c - n_r)^2 \end{aligned} \quad (1)$$

Here the nominal point value for the unranked candidates is $p_u = 0$ while v_u and $v_t(i)$ are modeled as indicated above for the specified point conversion values $p_t(i)$.

Likewise a weighted average of ballots $b \in I(s)$ of weight w_b is interpreted as a random vector X_s which is a mixture of random vectors X_b of mean rating vectors μ_b and variance vectors v_b . That is, $X_s = \sum_{b \in I(s)} w_b X_b$ with weight

$$w_s = \sum_{b \in I(s)} w_b \quad (2)$$

Then the maximum likelihood estimate for μ_s leads us to minimize $\sum_{b \in I(s)} (|\mu_b - \mu_s| / \sigma_b)^2 w_b$ to get, for each candidate c

$$\mu_s(c) = \frac{1}{\widehat{w}_s(c)} \sum_{b \in I(s)} \mu_b(c) \widehat{w}_b(c) \quad (3)$$

where

$$\widehat{w}_s(c) = \sum_{b \in I(s)} \widehat{w}_b(c) \quad (4)$$

and $\widehat{w}_b(c) = w_b / v_b(c)$. For strong ranking

$$I(s) = \{b : (b_1, \dots, b_n) = s\} \quad (5)$$

when the n top ranked candidates $b_1, \dots, b_n$ of ballot b are consolidated to s . In the case of weak ranking, s is determined by candidates $(c_1, \dots, c_n)$ with ranking levels $r_1 \leq \dots \leq r_n$ for $n \leq m_r$, with decreasing point values. It works the same way with ratings except that the point values become negative after finishing the positive rating levels.

The corresponding variance for X_s is given by

$$v_s(c) = \frac{1}{\widehat{w}_s(c)} \sum_{b \in I(s)} (v_b(c) + \mu_b^2(c)) \widehat{w}_b(c) - \mu_s^2(c) \quad (6)$$

For example, the maximum likelihood estimates of the global mean and variance, over all ballots, are:

$$\mu_g = \frac{1}{\widehat{w}_g} \sum_b \mu_b \widehat{w}_b \quad (7)$$

$$v_g = \frac{1}{\widehat{w}_g} \sum_b (v_b + \mu_b^2) \widehat{w}_b - \mu_g^2 \quad (8)$$

where $\widehat{w}_g = \sum_b \widehat{w}_b$.

The corresponding mean “noise level” for each candidate, $\widetilde{\mu}_g$ and its sigma $\widetilde{\sigma}_g = \sqrt{\widetilde{v}_g}$ are may be estimated by zeroing out the top ratings, for each ballot. For example, use only the bottom 2 or 3 candidates for each ranked ballot, or the 2 or 3 middle for a rating ballot. Then we compute the *correlation noise level* as

$$\mu_N = (\widetilde{\mu}_g + N_\sigma \widetilde{\sigma}_g) \cdot (w / \Sigma w) \quad (9)$$

for an appropriate *noise sigma factor* N_σ (normally, 1.30), using a weighted average with weights $w = \mu_g - \tilde{\mu}_g$. For ranking data μ_N is subtracted from each cluster mean vector when computing its correlation with another cluster mean vector, as in section 4.2. Second, when *portions* are computed for candidate support for a particular voting block, we use a *portions factor* P_f (normally, 0.50) to zero out a cluster's the mean vector below $P_f * \mu_N$ to reduce spurious proportionality, as in section 7.1. While big differences in μ_N produce big changes in the clustering results, small perturbations can fine tune the performance in a robust way. This we use to optimize the DTV match (section 9.3).

3.2 Consolidation to Slate Ballots

In the consolidation to slate ballots, the slates themselves are usually determined by the top 1, 2, or 3 candidates, plus a bottom ranked candidate or two in the case of ratings. The slate ballots weights are required to exceed a specified minimum weight m_w , typically yielding 50 to 150 slate ballots. Ballots which rank more than 3 candidates often lack sufficient weight, so they are combined with ballots with a smaller slate, an $n_r = 4$ combining with the $n_r = 3$ ballot of its top 3 candidates, for example. However, some ballots may end up combining with an $n_r = 1$ ballot, still without sufficient weight. These are designated as slate ballots despite their weight deficiency, so that the slate ballots constitute a full partition of the initial ballots.

For rating initial ballots the process is similar except that an initial ballot will first be combined with a ballot with a smaller set of negatively rated candidates, then with smaller sets of positively rated candidates, iterating until the minimum weight is exceeded or until both the positive and negative sets are of size 1. Unlike the ranking case, there may be many of the latter since, there are $n_c * (n_c - 1)$ positive/negative pairs for n_c candidates, often doubling the number of slate ballots and processing times, even though we limit the number of positively rated candidates and require that the number of negative candidates be less. The reason for the required emphasis on positive candidates is that only these may contribute to the portions that elect candidates (section 7.1).

4 Cluster Formation

The slate clusters are the members of the regular clusters, except that the slate clusters may be partially members of more than one regular cluster, permitting overlapping regular clusters. That is, these cluster sets may be regarded as *fuzzy sets* instead of ordinary sets. When using fuzzy sets, the memberships of all slate clusters not fully included in the set of regular clusters are represented by the set of *independents*, so that the partial memberships of each slate cluster in the regular clusters, plus the independents, add up to exactly 1.0.

4.1 Mean Cluster Vectors

We represent a regular cluster k as a fuzzy set of slate clusters s using a membership function m_{ks} . Again, we can interpret this as a random variable X_k which is a mixture of slate ballot random variables X_s , or $X_k = \sum_{s \in L_k} [m_{ks} w_s] X_s$, leading to the maximum likelihood minimization of $\sum_{s \in L_k} (|\mu_s - \boldsymbol{\mu}_k| / \sigma_s)^2 m_{ks} w_s$. At first glance, the mean rating vector $\boldsymbol{\mu}_k$ of cluster k is

$$\boldsymbol{\mu}_k(c) = \frac{1}{\widehat{w}_k} \sum_s \mu_s(c) [m_{ks} \widehat{w}_s] \quad (10)$$

where $\widehat{w}_s = w_s / \sigma_s^2$. Here actual cluster weight w_k and the normalized cluster weight vector $\widehat{w}_k$ are given by

$$w_k = \sum_s m_{ks} w_s \text{ with } \widehat{w}_k = \sum_s m_{ks} \widehat{w}_s \quad (11)$$

using the slate cluster mean vectors μ_s and weights w_s computed in (3). The corresponding variance vector is

$$\sigma_k^2 = \frac{1}{\widehat{w}_k} \sum_s (\sigma_s^2 + |\mu_s - \boldsymbol{\mu}_k|^2) [m_{ks} \widehat{w}_s] \quad (12)$$

Except that all this is actually an over simplification because because the membership function m_k depends on the mean vectors μ_k that we are attempting to compute, as detailed in the next section. That is, the problem is nonlinear, requiring an iterative, Newton-type solver, as developed in section 6.

4.2 Membership Function

The initial membership function for a regular cluster k is based on a modified correlation q_{ks} between k and a slate cluster s

$$q_{ks} = \Lambda_{ks} t_k \cdot t_s \quad (13)$$

where t_s is the normalized and centered mean vector $t_s = (\boldsymbol{\mu}_s - \mu_N) / |\boldsymbol{\mu}_s - \mu_N|$ for the cluster s , μ_N is the noise level (9) and

$$\Lambda_{ks}(c) = \begin{cases} \lambda & \text{if } t_k(c) < 0 \text{ and } t_s(c) < 0 \\ 1 & \text{otherwise} \end{cases} \quad (14)$$

for candidate c . Normally we set λ to 0 for ranking and to 0.5 for rating. The assumption is that negative correlations for ranking data are “in the noise” and should not contribute to a positive correlation, and this is critical to get good cluster merging, in addition to membership.

Then the initial membership function h_{ks} for slate cluster s in cluster k is defined from the modified correlation q_{ks} by

$$h_{ks} = C_r(q_{ks}) \quad (15)$$

where C_r is the rising cosine function

$$C_r(q_1, q_2, x) = \begin{cases} 0 & \text{for } x \leq q_1 \\ \frac{1 + \cos(y\pi)}{2} & \text{for } y = \frac{q_2 - x}{q_2 - q_1} \text{ and } q_1 < x < q_2 \\ 1 & \text{for } x \geq q_2 \end{cases} \quad (16)$$

for min and max correlation parameters q_1 and q_2 , such as $q_1 = 0.035$ and $q_2 = 0.70$.

This is scaled by a fuzzy set scale factor r_f to get the fuzzy membership function

$$m_{ks}(h_{\cdot s}) = r_f(S_p(h_{\cdot s})h_{ks}) \quad (17)$$

where, for each slate ballot s :

$$S_p(h_{\cdot s}) = \sum_k h_{ks} \quad (18)$$

over the regular clusters k that contain s . The fuzzy set concept is that the combined membership $S_p(h_{\cdot s})$ over the regular clusters with member s must

be scaled by a reduction factor $r_f(S_p)$ to make $S_p < 1$, as necessary, with the leftover $1 - S_p$ membership assigned to the independents. I define the scale factor r_f to provide a smooth transition instead of a sharp cutoff as S_p approaches 1:

$$r_f(S_p) = \begin{cases} 1 & \text{for } S_p \leq S_1 \\ < 1/S_p & \text{as } S_p \rightarrow \infty \end{cases} \quad (19)$$

using $S_1 = 0.90$, with the requirement that r_f be differentiable with $r'_f(S_1) = 0$. A suitable function is given by

$$r_f(S_p) = \frac{1 - (1 - S_1) d(S_p)}{S_p} \quad (20)$$

where for $S_p > S_1$

$$d(S_p) = \exp\left(-\frac{S_p - S_1}{1 - S_1}\right) \quad (21)$$

Note also that

$$r'_f(S_p) = \frac{d(S_p) - r_f(S_p)}{S_p} \quad (22)$$

with $r'_f(S) \sim -1/S^2$ asymptotically, from (20) and (21). Moreover for $x = q_{ks}$

$$C'_r(x) = \begin{cases} 0 & \text{for } x \leq q_1 \text{ and } x \geq q_2 \\ \frac{\pi \sin(y\pi)}{2(q_2 - q_1)} & \text{otherwise, for } y = \frac{q_2 - x}{q_2 - q_1} \end{cases} \quad (23)$$

5 The Clustering Objective Function

Our objective function for a cluster set begins with the primary objective, the *coherent* vote $V(\boldsymbol{\mu})$ of the converged regular clusters with mean vectors $\boldsymbol{\mu} = \{\boldsymbol{\mu}_k\}$:

$$V_c(\boldsymbol{\mu}) = \sum_k w_k P_D(k) P_B(k) \quad (24)$$

where the cluster size, or weight, w_k is given by (11) and $P_D(k)$ and $P_B(k)$ are small penalties for a low density or diffuse boundary for cluster k , given by (26) below. That is, the point of proportional representation is to categorize and count most voters into clusters, but with a preference for clusters that are larger and more coherent in the sense of being more dense and compact, while allowing for modest overlaps with other clusters.

For this purpose we used not only the rising cosine function (16) but also the falling cosine function:

$$C_f(q_1, q_2, x) = \begin{cases} 1 & \text{for } x \leq q_1 \\ \frac{1 - \cos(y\pi)}{2} & \text{for } y = \frac{q_2 - x}{q_2 - q_1} \text{ and } q_1 < x < q_2 \\ 0 & \text{for } x \geq q_2 \end{cases} \quad (25)$$

Then we define

$$P_D(k) = C_r(d_1, d_2, D_k) \text{ and } P_B(k) = C_f(b_1, b_2, B_k) \quad (26)$$

with typical values of $d_1 = 0.15$, $d_2 = 0.45$, and $b_1 = 0.10$, $b_2 = 0.35$. Here

$$D_k = \frac{\sum_{s \leq t} q_{st} w_{ks} w_{kt}}{\sum_{s \leq t} w_{ks} w_{kt}} \quad (27)$$

for slate clusters s and t both inside cluster k with $w_{ks} = m_{ks} w_s$ being the weighted membership of s in k for the fuzzy membership function m_{ks} (17) and slate cluster weight w_s (2). Likewise

$$B_k = \frac{\sum_{s,t} q_{s,t} w_{ks} (1 - m_{kt}) w_t}{\sum_{s,t} w_{ks} (1 - m_{kt}) w_t} \quad (28)$$

for s inside cluster k and t either fully ($m_{kt} = 0$) or partially outside k . These formulas generalize simple graphical density, which is just the ratio of the number of edges that connect the vertices of cluster to the maximum possible number of such edges, also simple graphical boundary, which is just the number of edges with one end in the cluster and the other end not in the cluster to the maximum possible number of such edges.

In addition we incorporate the size and correlation constraints into the objective function via the penalty functions P_W and P_C on the sizes w_k (12) of regular clusters and correlations q_{kl} (13) between them:

$$P_W = \min_k C_r(w_1, w_2, w_k) \quad \text{and} \quad P_C = \max_{k < l} C_f(o_1, o_2, q_{kl}) \quad (29)$$

Typical parameters are $w_1 = 0.25$, $w_2 = 0.75$, and $o_1 = 0.25$, $o_2 = 0.75$, noting that at the end of convergence, clusters with size penalty $P_W > 0.5$ are deleted and clusters with correlation penalty $P_C > 0.5$ are merged.

It often happens that a cluster set with only one regular cluster will have a rather large set of independents, with no other penalties, hence a stronger

clustering objective than a cluster set with more than one regular cluster, unless the large set of independents is penalized. Thus we define a penalty for the size w_i of the independents by

$$P_I = C_f(x_1, x_2, w_i) \quad (30)$$

with typical parameter values $x_1 = 0.40$ and $x_2 = 0.80$. The final penalty is for poor convergence:

$$P_R = C_f(e_1, e_2, \varepsilon_r) \quad (31)$$

where a normalized root sum of squares functional $\varepsilon_r(\boldsymbol{\mu})$ is being minimized, as in section 6.1, with typical limit values $e_1 = 0.025$ and $e_2 = 0.25$. That is, failure to fully converge is a sign of an ambiguous set of clusters. Another such sign is if only one of 20 initial cluster sets converges to a specific final cluster set, in which cases, we disregard this cluster set.

Then the overall *clustering objective* function $O(\boldsymbol{\mu})$ for a cluster set becomes

$$O(\boldsymbol{\mu}) = V_c(\boldsymbol{\mu}) P_c(\boldsymbol{\mu}) \quad (32)$$

where the *clustering penalty* is

$$P_c(\boldsymbol{\mu}) = P_W P_C P_I P_R \quad (33)$$

6 Cluster Set Computation

6.1 Convergence and damping

Symbolically, any set of candidate point vectors $\boldsymbol{\mu} = \{\boldsymbol{\mu}_k\}$ determines a set of possible converged clusters by computing the fuzzy membership m_{ks} of each slate cluster s in each provisional cluster k as centered about $\boldsymbol{\mu}_k$, as in section 4.2. Then these new membership values m_{ks} determine revised mean vectors $\boldsymbol{\mu}_k$, as in section 4.1, until the cluster set has converged to a fixed point. This convergence procedure is a generalization of the standard fuzzy k-means, or centroiding, algorithm[5], the first difference being our update formula for the membership values, the second difference being our cluster merging and deletion procedure, and the third difference being our damping procedures for the convergence. 20 initial cluster sets typically converge to only a few distinct final cluster sets, with one often being dominant.

During centroiding some of the clusters will tend to coalesce while others will remain isolated. Therefore a key strategy is to identify strongly connected pairs or groups of clusters and merge them into single clusters. We search for cliques of clusters - sets of clusters whose pairwise correlations exceed a specified correlation value q_m , first with a higher correlation value, later with lower values. After no more such mergeable cliques are found, then any small, isolated clusters are deleted.

The merging algorithm starts by separating out all singleton clusters - those which are isolated in the sense that their maximum correlation with all the other clusters is less than q_m . Then come pairs - two clusters connected by a correlation $> q_m$ only to each other, ready for immediate merging. Then the q_m - maximal cliques Q of size 3 or more. To list these maximal cliques Q , we use a standard multi-pass labeling algorithm for decomposition into connected components following by a standard depth first search, see ref [4]. From the list of the best maximal cliques in a component, we extract a maximal set of disjoint cliques and merge the clusters in each of them. Normally just the top clique in a component is used due to our relatively small number of candidates.

A common theoretical framework for such a problem is to formulate it as a system of equations to be solved, or as a functional to be minimized, using a Newton type method. In this case, if we let $U(\boldsymbol{\mu})$ denote the update operator for the iteration $\boldsymbol{\mu}^{n+1} = U(\boldsymbol{\mu}^n)$ we have

$$U_k(H, c) = \frac{1}{\widehat{w}_k(H, c)} \sum_{s \in L_k} \mu_s(c) [m_{ks}(H_s) \widehat{w}_s(c)] \quad (34)$$

for membership normalization factor $\widehat{w}_k(H, c) = \sum_{s \in L_k} m_{ks}(H_s) \widehat{w}_s(c)$, cluster set $\boldsymbol{\mu} = \{\boldsymbol{\mu}_k\}$ with $k = 1, \dots, n_\mu$, candidates $c = 1, \dots, n_c$, initial memberships $H_s = \{h_{ks}\}$, as in (15), $H = \{H_s\}$, with L_k = the set of cluster k 's member slate ballots.

Then the natural set of equations for the fixed point problem is

$$F(\boldsymbol{\mu}) = \boldsymbol{\mu} - U(\boldsymbol{\mu}) = 0 \quad (35)$$

with number of unknowns $n_u = n_c n_\mu$. Of course, when we can demonstrate that U is a good contraction operator (Jacobian < 1) we can simply iterate as in k-means. However it turns out that U may not always be a contraction operator in the neighborhood of $\boldsymbol{\mu}$, leading to slow or partial convergence, or wandering or divergence.

A standard test for such instability is to compute the L_2 norm of the operator, or spectral radius $\lambda(J) = \text{maximum absolute eigenvalue of the Jacobian matrix } J(\boldsymbol{\mu}) = \nabla U(\boldsymbol{\mu})$, to verify that $\lambda(J) < 1$. If not, the simple iteration $\boldsymbol{\mu}^{n+1} = U(\boldsymbol{\mu}^n)$ may be replaced by a damped Newton iteration to minimize the sum of squares functional $\frac{1}{2} \sum F^2(\boldsymbol{\mu})$. Taking the gradient yields the system of equations $E(\boldsymbol{\mu}) = \nabla F^t(\boldsymbol{\mu})F(\boldsymbol{\mu}) = (I - J^t(\boldsymbol{\mu}))F(\boldsymbol{\mu}) = 0$. The full Newton method for this system would require second derivatives: $(-\nabla J^t)F + H$ where $H = (I - J^t)(I - J)$. The simpler Gauss-Newton method uses only H , as in the normal equations. A damped version of this adds a parameter d to the diagonal so that

$$H_d(\boldsymbol{\mu}) = H(\boldsymbol{\mu}) + d I \quad (36)$$

$$\boldsymbol{\mu}^{n+1} = \boldsymbol{\mu}^n - H_d^{-1}(\boldsymbol{\mu}^n)E(\boldsymbol{\mu}^n) \quad (37)$$

In our case we do a quick line search over the damping parameter d . This approximates a gradient search to minimize the sum of squares residual for larger values of d while it is similar to the Gauss-Newton update for smaller values of d . If this update stagnates badly, we designate the convergence as failed and proceed to test convergence for the next initial set.

We use three *restart* phases for the convergence: Phase 1 is at the start or after a reduction in the number of clusters by merging, aiming only for partial convergence. Phase 2 follows phase 1 or after the number of clusters was decreased only by deletion, aiming for a medium level of convergence. Phase 3 follows phase 2, after all merging or deletion of clusters, aiming for good convergence. Typical convergence tolerances for the three phases are (0.10, 0.05, 0.01) for normalized Euclidean update $\sqrt{\sum F^2(\boldsymbol{\mu})/n_u}$. Merging and deletion criterion are tightened at each phase, with convergence failure declared only at phase 3. Typical cluster correlation criteria for merging at the 3 phases are (0.80, 0.65, 0.50), with deletion criteria for cluster size in position units as (0.20, 0.35, 0.50). 80% of 110 successful convergences without any damping, with 89% fully converged to the 0.01 level. 10 of the 40 districts had at least one failure of convergence, but this is OK since other initial sets did yield convergence.

Straight forward calculus gives an analytical formula, verified numerically by finite differences, for the Jacobian $\nabla U(\boldsymbol{\mu})$, given, for clusters k and l and candidates c and d by

$$\frac{\partial U_k(H, c)}{\partial \boldsymbol{\mu}_l(d)} = \frac{1}{\hat{w}_k(H, c)} \sum_{s \in L_k} (\mu_s(c) - U_k(H, c)) \hat{w}_s(c) \frac{\partial m_{ks}(H_s)}{\partial \boldsymbol{\mu}_l(d)} \quad (38)$$

Now

$$\frac{\partial m_{ks}(H_s)}{\partial \boldsymbol{\mu}_l(d)} = \sum_{j \in L_s} \frac{\partial [r_f(S_p(H_s)) h_{ks}]}{\partial h_{js}} \frac{\partial h_{js}}{\partial \boldsymbol{\mu}_l(d)} \quad (39)$$

where L_s is the set of clusters containing slate ballot s and for $j \in L_s$

$$\partial [r_f(S_p(H_s)) h_{ks}] / \partial h_{js} = r'_f(S_p(H_s)) h_{ks} + r_f(S_p(H_s)) \delta_{jk} \quad (40)$$

Next

$$\frac{\partial h_{js}}{\partial \boldsymbol{\mu}_l(d)} = \delta_{jl} C'_r(q_{js}) \frac{\partial t_{j\lambda} \cdot t_s}{\partial \boldsymbol{\mu}_l(d)} = \delta_{jl} C'_r(q_{js}) \sum_c t_s(c) \frac{\partial t_{j\lambda}(c)}{\partial \boldsymbol{\mu}_l(d)} \quad (41)$$

where

$$\frac{\partial t_{l\lambda}(c)}{\partial \boldsymbol{\mu}_l(d)} = \frac{\partial z_{l\lambda}(c) / |z_{l\lambda}|}{\partial \boldsymbol{\mu}_l(d)} = \Lambda_{ls}(d) (\delta_{cd} - z_{l\lambda}(c) z_{l\lambda}(d) / |z_{l\lambda}|^2) / |z_{l\lambda}| \quad (42)$$

so that

$$\frac{\partial h_{ls}}{\partial \boldsymbol{\mu}_l(d)} = C'_r(q_{ls}) \Lambda_{ls}(d) (t_s(d) - q_{ls} t_{l\lambda}(d)) / |z_{l\lambda}| \quad (43)$$

6.2 Cluster Set Initialization

Our convergence algorithm for cluster sets requires initialization from a variety of plausible or random initial cluster sets in order to not miss out on optimal or near optimal solutions to the global cluster optimization problem. We've tested a small selection of typical sub-optimal clustering methods (5), along with a larger number of random selections based (15), all on the slate clusters. For the latter we combine slate clusters into many *generalized candidate clusters* - those which top rank a given set of candidates, then choose a subset of these at random, typically of size 7.

One kind of conventional clustering algorithms is to compute the "clustering coefficients", such as in ([6]) or ([7]). The graph in question has the slate clusters as weighted vertices, with edge weights given by correlations above a certain cutoff. Then, for initialization, we use the mean vectors of the vertex

peaks of this graph, based on their degree or a triangular generalization of degree. Increasing the correlation cutoff increases the number of peaks. We also tested a *core* clustering method, where each cluster is represented by a core, or clique, of strongly correlated vertices, not just a central peak. In addition we extended the STV and DTV algorithms, described in the next section, to produce cluster sets to initialize the convergence.

6.2.1 Extended STV and DTV

STV, as is, yields only rudimentary candidate clusters - one centered on each elected candidate, with the pseudo-cluster of *independents* containing all the remaining full or partially weighted ballots. The only overlap is when a ballot representing a newly elected candidate has part of its weight transferred to the candidate cluster of its next ranked *hopeful* candidate - one still eligible for election. That is, at each STV cycle, a hopeful candidate is either elected or eliminated. A candidate is elected if its candidate cluster has the most weight and exceeds a quota, given by the Droop formula below. If none exceed the quota, then the candidate with the least weight candidate cluster is eliminated. A hopeful candidate cluster at this cycle consists of all ballots which top rank the candidate after ignoring all prior elected and eliminated candidates, with the ballot weight reduced by the transfer factor associated with each higher ranked elected candidate.

When a candidate is eliminated, the total weight of that ballot, which is initially the number of original identical ballots that it represents, is transferred to the next ranked *hopeful* candidate. This is the same as in IRV. But when a candidate is elected, only part of its weight is transferred to the next hopeful candidate. The transfer fraction that determines this part is computed so that the sum of the retained weights of all the ballots in the candidate cluster equals the quota. The Droop formula for the quota is $Q = (\sum_h W_h) / (n_r + 1)$ where W_h is the weight of the candidate cluster for the hopeful candidate h and n_r is the number of candidates remaining to be elected. This quota remains the same after a candidate is elected but decreases after an elimination.

After the required number n_p of candidates are elected, the standard STV algorithm stops but our extended STV continues, first finishing the transfers of the last elected candidate, then all the remaining undeleted candidates as they are eliminated. A good many of the ballots will still have some

weight which was not incorporated into the quotas of any of the elected candidates. But the requirement for transferring to the next hopeful candidate no longer makes sense since it will not help any other candidate to get elected. Thus we make the transfer to the next ranked candidate, whether already elected or still undeleted. The result is that the sizes of the elected candidate clusters grow well beyond their quota values, with prior transfers from elected candidates being at least partially reversed, giving a more realistic clustering in terms of cluster size, though not yet in cluster overlap.

The second stage of our *extended* STV is to convert rankings to points, using our modified Borda (section 2). Apply this to the ballots in each first stage cluster to get a mean rating vector for that cluster. Next compute the correlations between these mean rating vectors. Finally, apply a cluster merging algorithm based on these correlations. That is, if a correlation exceeds 0.5, then merge the 2 clusters by taking the union of their member ballots, adding the weights of the ballots in their intersection. A reasonable way to do this, is to order the clusters from smallest to largest in size and test the merger criterion for the smallest with each larger cluster. Whenever there is a merger, recompute the relevant correlations and reorder the clusters, then repeat.

However, the largest remaining fraction or D'Hondt methods (section 7.3) applied to these cluster sizes and mean rating vectors may differ from the original STV electeds, showing the limitations of even the second stage of extended STV, though it remains valid as an initial cluster set for convergence. The first step of this convergence is to compute the cluster memberships consisting of slate clusters by correlating these with the mean rating vectors of the extended STV clusters and applying the fuzzy set criterion so that the sum of the memberships of a slate cluster over the STV clusters does not exceed 1. This could be viewed as the third stage of extended STV, with the converged result as the fourth stage. However, sometimes the STV elected set may yield a somewhat higher full objective value when paired with a different converged cluster set, a typical result of complexity.

Distributed Transferrable Vote (DTV) is a substantial improvement to ordinary STV. Here the transfer fraction of weight from an elected or eliminated candidate is the same but this fraction is distributed, not to the top hopeful, but to the ranked list of hopefuls, or electeds + hopefuls in extended STV, according to a declining fraction. Thus a typical distribution over a list of 4 eligible candidates in ranking order would be (0.467, 0.300, 0.167, 0.067), which comes from our modified Borda (section 2). To be effective,

this distribution must start at the very beginning of DTV, when all candidates are hopeful, with the full weight of each ballot being distributed to all its ranked candidates. The result is a better clustering because it fully utilizes all ballot rankings.

The Warren variation ([9], p 274-5), on STV changes how fractions of ballot weights are to be retained by a candidate c being elected. The effect is to fully retain ballots of small weight, while retaining a fixed weight w_r from ballots of larger weight, while requiring that the total retained be the Droop Quota Q , with the rest transferred to surplus S as in STV. That is, if the n ballots b for candidate c are ordered by increasing weight w_b , then the m ballots up to a weight w_r will be fully retained with w_r retained as a fixed value from all larger ballots. Here m is increased until $w_r = (Q - \sum_1^m w_b) / (n - m)$, where $S = \sum_{m+1}^n (w_b - w_r)$ and Q has been reduced by non-transferable ballots. Over our 40 districts this reduced the clustering mismatches from the STV value of 11 to 10, with a big reduction in the number of ballot transfers and computational complexity. However, the argument between the traditional Meek method and the Warren method has depended more on economic interpretations.

7 Candidate Matching

The next issue is how to match candidates to sets of clusters. Ideally each cluster set rates highly a different set of candidates, so that if a cluster represents a fraction x of the ballots, then it gets to elect approximately $x n_p$ of the candidates, starting with its top rated candidate. In practice these fractional values must, of course, be translated into whole numbers in some reasonable way. Also, some candidates may be highly rated by more than one cluster, leading to more complications.

7.1 Matching Objective

Let us start by asking, "How does each voting block rank or rate the candidates?" That is, who do they want to represent them. Our answer is to use the mean rating vector μ_k of the voting cluster k , the higher the rating, and larger the cluster size, the stronger the representation. This notion may be quantified by the concept of a *portion* defined below. However this portion, if for a regular cluster, should be zero unless the candidate has a

sufficiently positive rating by the cluster. Otherwise candidates who are indifferent to a voting block could contribute to its representation, thus influencing proportionality in inappropriate ways. Instead, the intention is that the pseudo-cluster of independents should incorporate such influences. This is also important for the outcome since sometimes the only good proportional match of candidates to clusters is when a candidate represents the independents, which does not have a proportionality mismatch penalty. This is also critical computationally. For this purpose we start by zeroing out half the high noise value μ_N that works better for correlations (9):

$$\mu_{kz}(c) = \mu_k(c) \text{ except } \mu_{kz}(c) = 0 \text{ where } \mu_k(c) \leq \mu_N/2 \quad (44)$$

For each cluster, it is useful to have a way of identifying four kinds of candidate that could at least partially represent that cluster: (1) top candidates, (2) significant but secondary candidates, (3) still viable but more marginal candidates, and (4) the indifferent or hostile candidates. A simple way to get the top candidates is to sort the cluster mean vector μ_{kz} into decreasing order to look for the last candidate n_1 such that $\mu_{kz}(n_1) > 0.67 \mu_{kz}(1)$. Then search further for the last candidate n_2 such that $\mu_{kz}(n_2) > 0.25 \mu_{kz}(1)$ to add significant candidates, with the last candidate n_3 with $\mu_{kz}(n_3) > 0$ adding any remaining viable candidates, with the indifferent or hostile candidates already zeroed out.

The concept of *portion* allows us to compute the actual fractional number of candidates $A_k(E)$ (47) for a possible set of electeds E to represent a cluster k . We define the *portion* $p_L(c, k)$ of candidate c that represents cluster k of cluster set L , including independents, by

$$p_L(c, k) = \frac{\mu_{kz}(c) w_k / n_p}{\mu_{Lz}(c)} \quad (45)$$

where the cluster sizes w_k are scaled so that $\sum_k w_k = n_p$ and $\mu_{Lz}(c)$ is the *cluster-averaged zeroed mean vector* μ_{kz} of (44) for candidates c over all clusters k , including independents, weighted by the cluster sizes w_k :

$$\mu_{Lz}(c) = \left(\sum_k \mu_{kz}(c) w_k \right) / n_p \quad (46)$$

so that $\sum_{k=1}^{n_i} p_L(c, k) = 1$ for each candidate c .

To get proportional representation, our goal for a set of candidates E is that it should satisfy $A_k(E) \cong w_k$ where $A_k(E)$ is the actual fractional number of candidates from set E that represents a voting cluster k :

$$A_k(E) = \sum_{c \in E} p_L(c, k) \cong w_k \quad (47)$$

Examining all possible sets of electeds E , subject to this soft constraint, we should choose the set with the largest average rating, the *elected rating* function

$$\mu_{Lz}(E) = \left(\sum_{c \in E} \mu_{Lz}(c) \right) / n_p \quad (48)$$

Thus the optimal set E is the subset that combines the *elected rating* $\mu_{Lz}(E)$ with good proportionality (47), which we quantify by defining a *cluster size*, or *proportionality penalty* function

$$P_L(E) = \prod_{k \in L} P_k(E) \quad (49)$$

where $P_k(E) = C_f(d_1, d_2, d_k)$ is a penalty function using the falling cosine function (25), and $d_k = |A_k(E) - w_k|$ is the deviation from proportionality, for cluster k . Typical deviation limits are $d_1 = 0.20$ and $d_2 = 0.70$ fractions of a position when proportionality is interpreted as minimizing the worst absolute deviation, independent of cluster size. Except there is no penalty here for the pseudo-cluster of independents, which has a separate penalty for very large sizes without regard to the set of electeds (30).

Then we define the *fitness* of the set of possible elected E for the cluster set L as the product of the *elected rating* and *proportionality penalty*:

$$M_L(E) = \mu_L(E) P_L(E) \quad (50)$$

7.2 Assignment Algorithm

To find the best set of elected candidates E for a given cluster set L , we could compute the objective function $M_L(E)$ (50) over all $\binom{n_c}{n_p}$ possible subsets E of size n_p . But in practice we apply a branch and bound algorithm ([5], pp. 141-143) which adds successive candidates to a partial elected set E in a depth-first search, keeping E in increasing order.

Our bounding, or pruning, procedure, is that at each stage of this search a subset E of potential electeds, may be extended to a full set without affecting proportionality by adding the first candidate after n_0 , which is the last candidate with $p_L(c, i) < 1$ for independent set i . That is, $p_L(c, k) = 0$ for each regular cluster k for subsequent candidates c in the decreasing ordering of $\nu_{Lz}(c_{n_0})$:

$$\nu_{Lz}(c) = \mu_{Lz}(c)(1 + \varepsilon - p_L(c, i)) \quad (51)$$

where μ_{Lz} is the cluster-averaged zeroed mean vector (46).

For example, if $E_0 = \{c_1, c_4, c_5\}$ with $n_0 = 5$ and $n_p = 4$, then if E_0 is extended to $E_x = \{c_1, c_4, c_5, c_6\}$ it will have the same actual size $A_k(E_x)$ for each regular cluster k but higher electeds rating $\mu_L(E_x)$ compared to any other extension E_y since the ν_{Lz} ordering will be the same as the μ_{Lz} ordering in this region. Typically, we compute the top 25 possible sets of electeds E , ordered by fitness $M_L(E)$, adding, for comparison purposes, the STV and DTV elected sets.

7.3 Extended D'Hondt

For another comparison we extend the election method of *D'Hondt* (see Wikipedia, for example) to apply to overlapping voting blocks, whose sizes and cluster mean rating vectors are known, including for the independents. This extension also applies to the largest remaining fraction and many other traditional methods. *D'Hondt* did a slightly better overall than remaining fraction but not consistently.

Basic D'Hondt only gives the whole number of representatives to be assigned to each cluster. *Extended D'Hondt* elects the individuals who will fill these slots, based on the candidate rankings provided by the cluster mean vectors. Normally a cluster with 2 slots would get its top and second ranked candidates elected, but what if another cluster has one of those as its top ranked? For this purpose we order the regular clusters from smallest to largest in size and assign candidates to the smallest first, figuring that the larger voting blocks are more likely to have a larger selection of well-rated candidates to choose from.

We do this in a sweep, starting with the top rated candidate in each regular cluster. Thus if a cluster's top choice was already chosen by a smaller cluster, the current cluster gets its next available choice. The second sweep is the same but for second choice candidates and applies only to clusters with 2

or more slots, etc. At the end any candidates allotted to the independents are assigned based on the order of the independents mean rating vector, skipping over any candidates already assigned.

8 Full Optimization

The best overall solution would seem to be the best set of elected candidates for the best set of clusters. But what if there is a second best set of clusters L that is almost as good as the best set, but with a set of electeds E that gives a significantly better *fitness* $M_L(E)$ (50)? That is, an overall ranking of possible sets of electeds needs to balance the *clustering objective* function $O(L)$ (32) against its elected set *fitness* $M_L(E)$. It's possible that a fractional increase in one factor would sometimes be balanced by a similar fractional decrease in the other factor, so we adopt the formula

$$F(E) = \max_L \{O(L) \cdot M_L(E)\} \quad (52)$$

as the *full objective* function $F(E)$ to determine the overall best elected set. Of particular interest is the ratio

$$R_t(e) = F(e)/F(1) \quad (53)$$

of the full objective of possible elected set e to that of the top elected set 1. Here the top tier max function is over all cluster sets L whose clustering objective $O(L)$ is $> 0.85 O(L_1)$ where L_1 is the top cluster set. A second tier goes down to $0.50 O(L_1)$, then all the rest. Controversies would be most likely to arise in the top tier, although a set of electeds E in the second tier might be considered competitive if the top tier sets were all unexpected or controversial. It's also possible that the top full objective comes from the bottom tier, but this is rejected due to the strong inferiority of its cluster set analysis.

9 Results

Over all 40 districts, there were 64 unique converged cluster sets = 12 + 32 + 16 + 4 with 1, 2, 3, or 4 regular clusters, or voting blocks. The best converged cluster set for each district counted (4,23,13,0) districts with regular cluster counts of (1,2,3,4). Sensitivities to many algorithm parameters were tested extensively, with several results illustrated below.

Table 1: Mean Vectors

Mean Vectors	1	2	3	4	5	6	7	8	9	10
Averaged	4.111	3.961	3.229	3.588	2.014	2.013	1.859	1.548	1.208	0.89
Cluster 1	7.284	6.985	0.0	5.450	0.0	0.0	0.0	0.0	0.0	0.0
Cluster 2	1.807	1.662	9.718	1.501	2.694	2.638	2.139	1.801	0.0	0.0
Independents	2.709	2.687	2.223	3.198	3.363	3.394	3.308	2.743	2.966	2.18

Table 2: Portions

Size	Portions	1	2	3	4	5	6	7	8	9	10
1.414	Cluster 1	0.626	0.623	0.0	0.537	0.0	0.0	0.0	0.0	0	0
0.957	Cluster 2	0.105	0.100	0.720	0.100	0.320	0.313	0.275	0.278	0	0
1.629	Independents	0.269	0.277	0.280	0.363	0.680	0.687	0.725	0.722	1	1

9.1 An Example of STV Failure

This example is from the Eastcentre district of the Glasgow elections of 2007, with 4 positions and 13 candidates narrowed to 10. All 20 of the initial sets converged to 1 cluster set with 2 regular clusters. STV elected the candidate set (1,2,3,4) whereas clustering elected (1,2,3,7). This is because the latter has better proportionality (50), which raised its fitness, hence also its full objective.

Table 1 displays the mean vectors for the clusters, using a noise zeroing cutoff of 1.436 (44).

These values determine the cluster *portions* (45) in table 2, then the proportionality for viable sets of electeds. That is, the zeroed ratings are averaged over the cluster mean vectors, weighted by cluster size, to give the “Cluster-averaged” row. Then these overall ratings are averaged over each feasible set of elected candidates to get its elected rating (48). Note that the cluster sizes sum to the number of electeds n_p , while the portions sum to 1.0 for each candidate, with each portion representing the strength with which the candidate is represented by each cluster:

The clustering objective is the product of the coherent vote and several penalty factors (1 = no penalty, 0 = full penalty). The table 3 gives the data for the coherent vote (24), which is mostly the total vote by the regular clusters, with occasional small penalties for clusters k which are a little too

Table 3: Coherent Vote

-	Size	Penalty	D_k	B_k
Cluster 1	1.414	1.00	0.533	0.056
Cluster 2	0.957	1.00	0.580	0.031
Coherent vote	5.927	-	-	-

Table 4: Clustering Objective

-	Product	P_W	P_C	P_I	P_R
Data	-	0.957	0.0	0.407	0.0002
Penalty factor	0.999	1.0	1.0	0.999	1.0
Clustering Objective	5.923	-	-	-	-

scattered; that is, not very dense (D_k), or with a diffuse boundary (B_k). The idea is that the clustering should maximize this vote subject to proportionality restrictions, with the independents playing a backup or secondary role. The value of the coherent vote is scaled by 10 / 4 to align with our rating scale.

The overall clustering objective is the coherent vote times the clustering penalty (32), which is the product of several penalty factors (33). In table 4 these penalty factors are the minimum weight of the regular clusters (P_W), the maximum correlation between them (P_C), the size of the independents as a fraction of 1 (P_I), and the convergence functional (P_R)

In table 5 $E_c = (1,2,3,7)$ is the clustering set of electeds, whereas STV and all other non-clustering methods elect $E_n = (1,2,3,4)$. The deviations are from proportionality of the actual cluster sizes $A_k(E)$ (47) for elected set E and cluster k , with penalty $P_k(E)$ (49). For example, $A_1(E_n) = 0.626 + 0.623 + 0.537 = 1.786$ and $w_1 = 1.414$, so $A_1(E_n) - w_1 = 0.372$, approaching a half candidate error, hence the growing penalty of 0.734. By contrast $(1,2,3,7)$ omits the 0.537 term, yielding a much more modest candidate position error of $-(1.414-1.249) = -0.165$, which has no penalty with our 0.20 cutoff value. Note that E_n is more proportional for cluster 2 than E_c but that the E_c penalty is not enough to make any difference.

Now table 6 demonstrates the impact of the disproportionality of cluster 1 on the objective functions.

Note that the STV electeds E_n do yield a significantly higher average rating but this is overcome by the strong disproportionality for cluster 1. A step-

Table 5: Proportionality

Proportionality	E_c deviation	E_c penalty	E_n deviation	E_n penalty
Cluster 1	-0.165	1.00	+0.372	0.734
Cluster 2	+0.244	0.979	+0.068	1.00

Table 6: Elected E_c vs E_n

Electeds	Clustering obj	Elected rating	Dev penalty	Fitness	Full obj
E_c	5.923	3.290	0.981	3.229	19.121
E_n	5.923	3.722	0.734	2.731	16.174

by-step examination of STV for Eastscentre shows that it doesn’t even come close to capturing this disproportionality, because it does not show how the single-candidate clusters of STV overlap and merge. However, if the bottom candidate is deleted from the computation, then E_c loses its edge over E_n , even more so if the bottom 2 candidates are deleted. Thus even lesser candidates can sometimes have an effect on the disproportionality outcome, but section 9.4 shows that the statistical differences are in line with other sensitivities.

9.2 Comparison with Non-clustering Methods

A comparison of *PR clustering* results compared to the standard STV method shows that STV elected the same set of candidates 28 times out of the 40 districts, with DTV even better at 34 exact matches.

In table 7 we compute the top *full objective* $F(e)$ value for each non-clustering elected set, maximized over all converged cluster sets in each district, and compute the ratio $R_t(e) = F(e)/F(1)$ with the top clustering *full objective* $F(1)$, as determined in section 8. We interpret a ratio > 0.90 as being very good. What is notable in the table is the perfect match of DTV to *PR clustering* for 34 out of the 40 districts, with Borda doing better than IRV and the Top Vote and as good as STV. However STV still does significantly better when there are mismatches, using the ratio $R_t(e)$ than all the other non-clustering methods, except for DTV. Thus DTV comes out as the best competitor to *PR clustering* but the surprise is that Borda is as good as STV for exact matches. This is because it yields the set with the highest cluster-size averaged rating (48), which is the winner when the cluster size disproportionalities $d_k(49)$ are modest. The mean ratio of 0.716 for the

Table 7: Optimized Non-clustering Success

$R_t(e)$	STV	DTV	IRV	Top	Borda	Condorcet	Approval
< 1.00	12	6	15	16	12	12	13
< 0.90	3	1	4	5	6	8	7
< 0.50	0	0	1	2	3	4	3
< 0.15	0	0	0	0	1	1	1
very good frac	0.925	0.975	0.900	0.875	0.850	0.800	0.825
mean ratios < 1	0.922	0.952	0.895	0.853	0.716	0.650	0.727

Table 8: STV vs DTV

Extended	# ratio < 1.0	# ratio < 0.9	# ratio < 0.5	very good	mean < 1
STV	14	7	2	0.825	0.807
DTV	10	6	1	0.850	0.796

mismatched districts shows how hard Borda can be hit by serious disproportionality, though it does better than Condorcet and has the big advantage of simplicity.

However, unlike the other non-clustering methods, we have extended STV and DTV into clustering methods, so we can compare their clustering performance by using the cluster sets they converge to, instead of whichever cluster set gives their best full objective. Then the first 2 columns of table 7 become the 2 rows of table 8 for the converged extended STV & DTV cluster sets.

What this drop in performance shows is that the good results in the first table are in part due to the fact that any proposed set of electeds will tend to achieve a higher objective when it can shop around to several different cluster sets, searching for the one with the best fit. In addition, the extended STV and DTV cluster sets come up short compared to optimized clustering.

9.3 Parameter Sensitivity

A key issue are elections which yield similar *clustering objective* values but different divisions into voting blocks. Just like ties, or near ties, in ordinary voting, these will happen sometimes. Then the outcome will likely depend on the choice of parameter values or other minutiae of the algorithm, like the sensitive dependence on initial conditions in complexity theory. How likely are such results and how severe are the consequences? In particular, how

Table 9: Δ Sensitivity

Parm 1 Δ	Frac match	Clust Obj	Fitness	DTV Ratio	Converged STV
0.200	0.625	-1.600	+0.374	-0.048	+0.007
0.400	0.800	-0.254	+0.084	-0.018	+0.009
0.450	0.825	-0.136	+0.030	-0.009	+0.001
0.500	1.000	7.215	4.575	0.993	0.932
0.550	0.975	-0.085	-0.037	-0.001	-0.001
0.600	0.975	-0.153	-0.072	-0.002	-0.001
0.800	0.950	-0.305	-0.192	-0.009	0.004

sensitive is the selection of the elected candidates, given the approximations and tradeoffs involved in the proportionality requirement?

To test sensitivity to perturbations in parameter values we first selected a key parameter from an early phase of the algorithm, one with a big effect on the mean vectors of the clusters. We tested 6 perturbations for each over the 40 districts against several measures. Our first measure “Frac match” is the fraction of districts where the top set of electeds of the perturbation run matches that of the standard clustering run. The second measure “Clust Obj” is the mean of the *clustering objective* (32) of the perturbed runs minus that of the standard run. The third measure “Fitness” is for the corresponding mean of the electoral *fitness* (50). The fourth measure “DTV Ratio” is for the ratio of the best full objective over all cluster sets for the DTV electeds to the PR clustering full objective. The fifth measure “Converged STV” is the ratio of the full objective for the STV electeds, assigned to the cluster set converged from the extended STV cluster set (section 6.2.1), to the optimal full objective. This STV ratio will often be smaller than the best full objective STV ratio.

The selected sensitivity parameter specifies the increase Δ in the increment of the modified Borda point values, ranging from 0.2 though 0.8, as described in section 2. This parameter reflects voter preference for how strongly point ratings should increase with the ranking.

Note that only the largest parameter deviations in table 9 have much effect, a good sign of the general robustness and stability of the algorithm. Otherwise there is only one trend - the steady decreasing fitness as the Δ parameter increases. This parameter increase causes the point values of the rankings of a ballot to decrease more rapidly for its lower ranked candidates,

Table 10: Original Non-clustering Success

$R_t(e)$	STV	DTV	IRV	Top	Borda	Condorcet	Approval
< 1.00	14	9	17	19	14	16	15
< 0.90	6	4	5	6	8	10	9
< 0.50	1	1	0	1	3	4	3
< 0.15	0	0	0	0	0	0	0

putting most of the point value on the top ranked candidates. This in turn decreases the averaged *elected rating* factor (48) of our fitness objective function. The high average DTV and STV ratios reflect the fact these ratios are 1 for most districts, more so for DTV.

The table 10 came from the μ_N statistical formula (9) with no optimization. Compared to the optimized table 7, table 10 shows that the DTV optimization gave a definite improvement in both the # of perfect matches, from 31 to 34 districts and in the # good ratios, from 36 to 39. Not surprisingly, the DTV optimization improved STV somewhat as well, with the other non-clustering methods similar or slightly better. Thus DTV started out in the lead and became even stronger with optimization, but this brought *PR clustering* a little closer to the other non-clustering methods as well, demonstrating its overall value and robustness.

Another sensitivity is from the proportionality penalty parameters $(d_1 d_2) = (0.20, 0.70)$ of (49) involved in matching possible elected sets to cluster sets. Decreasing the penalties for disproportionality will make even approximately proportional methods like STV and DTV look better, but the whole point of PR clustering is to include a reasonable penalty for disproportionality. That is, the sensitivity is to be expected, with the parameters chosen being reasonable on both theoretical grounds and by testing. The theory is that exact proportionality is impossible while there should be a strong penalty for serious disproportionality - a mismatch of more than $1/2$ of one elected seat. That is, mismatches of $< 1/4$ should be regarded as normal, with little or no penalty, while our cosine penalty functions make the objective functions smoother. For example, one could argue for a cliff (full penalty) at $1/2$ but this would force too big and sudden a penalty at 0.40.

All this suggests that proportional voters should not too concerned about the details of the algorithm in practice, at least as long as political parties and other interest groups are restricted from unethical manipulations.

9.4 Domain Sensitivity

The point here is that the top elected set could differ critically when it is computed for a domain of candidates that is a proper subset of the domain D_0 of size n_c of all candidates. To measure this sensitivity, we follow up on an idea of Tideman ([9], p 279-85), for a tournament that would pair one possible elected set against another on a variety of domains. For example, we could consider as *sensitivity domains* all $n(n+1)/2$ subsets D_i of D_0 of size $n_c - 2$. Pairs of possible elected sets s_1 and s_2 of size n_p are compared for each such domain D_i provided that $s_1 \cup s_2 \subseteq D_i$. A related possibility is to consider *union domains* = all subsets of size $n_p + 1$ to $2 * n_p$ with the requirement that $s_1 \cup s_2 = D_i$.

Our clustering algorithm automatically orders all potential elected sets by their full objective (50) for the top cluster set for each domain, yielding an optimal comparison. So the issues become the (1) choice of domains and (2) how to compare between domains. It turns out that there is a very simple rule - the *top set rule* - for the second issue. That is, choose the set s which is the top elected set by the objective comparison for the most number of domains $N(s)$, since each domain of size n will have $\binom{n}{n_p}$ subsets s of size n_p ordered by their *fitness* for its top cluster set. This works very well in practice, but a more nuanced measure of dominance would utilize the ratio of fitnesses: $R_i(s_1, s_2) = F_i(s_1)/F_i(s_2)$ for $s_1 \cup s_2 \subseteq D_i$. For the *dominance* of s_2 over s_1 , we use the rising cosine formula (16)

$$D_m(s_1, s_2) = C_r(-u, u, 1 - R_i(s_1, s_2)) \quad (54)$$

where $R_i(s_1, s_2) < 1$ with $D_m(s_2, s_1) = -D_m(s_1, s_2)$, using a suitable value for u , such as 0.15. The count $N(s_2)$ usually comes from dominance values $D_m(s_1, s_2)$ close to 1, but sometimes the final contest may be between sets which are very competitive - with dominance values closer to 0. For this purpose we compute the summed dominance value

$$S_d(s_2) = \sum_s D_m(s, s_2) \quad (55)$$

. For sensitivity domains D_i , the number of districts where the summed dominance S_d matches (PR clustering, STV, DTV, IRV, TopVote, Borda, Condorcet, Approval) on D_0 is (30, 28, 30, 28, 25, 25, 22, 24). For the top set rule the district counts are very similar: (31, 28, 31, 28, 25, 26, 23,

Table 11: Domain Comparisons

elected set	1,2,3,7	1,2,3,8	1,2,3,9	1,2,3,6	1,2,3,5	1,2,3,4	# dom	# top
nc=10 PR obj	19.12	18.62	18.52	18.44	18.23	16.17	1	1
nc= 9 PR obj	20.74	19.93	20.13	19.55	19.32	20.77	1	1
nc= 8 PR obj	18.30	17.89	-	17.21	17.37	26.91	1	1
nc=10 union S_d	34.66	30.08	34.71	34.59	34.62	29.04	627	174
nc= 9 union S_d	17.47	14.95	17.23	17.55	17.51	14.12	255	95
nc= 8 union S_d	8.04	5.73	-	8.07	8.20	8.06	93	46
nc=10 sens S_d	-	-	-	6.37	6.55	10.06	45	16
nc= 9 sens S_d	6.43	-	-	6.66	6.59	-	36	16
nc= 8 sens S_d	-	-	-	3.88	4.13	2.75	28	17

25), so the formula (54) only occasionally makes a difference. Thus the random removal of 2 candidates didn't change the final set of electeds 75% to 80% of the time for clustering and DTV and was 60% to 70% for the other non-clustering methods. This is certainly quite reasonable.

The summed dominance S_d for union domains yield district counts of (29, 26, 29, 26, 25, 24, 21, 25) for the tested methods, with the top set rule producing (33, 31, 34, 29, 26, 29, 25, 28). The latter shows that the much larger number of union domains seem to magnify the effectiveness of the top set rule, matching the effectiveness of DTV on D_0 but not PR clustering itself. It should be noted that the union domains, by definition, utilize one feature of STV, that is borrowed from IRV, which is the elimination of extraneous candidates from a comparison of 2 possible elected sets, but leaving the proportionality to be determined.

This domain sensitivity plays out in an interesting way for the East-centre election examined in section 9.1. PR clustering elected the set of candidates {1,2,3,7} with full objective 19.12 versus 16.17 for {1,2,3,4} but it turns out that several other sets were close competitors, and these varied somewhat when the number of candidates was reduced from 10 to 9, then to 8, with the next 3 rows being for union domains, and the last 3 rows for sensitivity domains.

Note that the PR clustering top set $E_c=\{1,2,3,7\}$ was first for rows 1 and 3, second for rows 2 and 4, third for row 5, and fourth for row 6. Whereas the standard non-clustering elected set $E_n=\{1,2,3,4\}$ was first for rows 2 and 3, third for row 6, and last for rows 1 and 5. In addition {1,2,3,9} was first

for row 3. This is an interesting result of complexity, noting that removing a candidate ends up combining some ballots and may change the portions enough to effect the outcome. For the domain sensitivity rows, some of the competitive elected sets were not the top set for any domain, depending on which 2 candidates were deleted. Thus the union domains produced better matches overall to the PR clustering, though with a far larger computational cost, as indicated by the # domains used in the computation.

10 Conclusions

Modern methods of scientific computing are well suited for certain classical problems of democracy, such as voting by proportional representation. Such algorithms may yield not only more optimal performance than classical methods like STV, even with significant improvements such as DTV, but also give insight into problematic voting circumstances. That is, tradeoffs can be identified, also sensitive dependence on certain aspects of the electorate, as well as of the algorithms. Both political scientists and strategists will have more to think about.

With further testing and standardization, our code, or a variant, could be used routinely in electoral circumstances where governments or citizen groups want both a better understanding of electoral outcomes or more control over the principles and parameters of the voting method to meet goals like proportionality. However this also requires a tolerance not only for the complexity of the algorithm but also of the intrinsic complexity, ambiguity, and paradoxes of many of the electoral conditions that will be encountered in practice. Public education and extended research are required, plus support from key political entities, not campaigns for immediate adoption of this or that method.

A somewhat surprising result was that the Borda set of electeds, not intended to be proportional, was as good as STV for exactly matching the set elected by *PR clustering*, matching our optimized clustering 28 out of 40 districts (7), or perhaps the surprise is that STV did not do better. The fact that DTV matched 34 out of 40 suggests the latter. In fact both our Borda with a 0.5 point increment (section 2) and DTV (section 6.2.1) distribute ballot points or weights over the top several ranked candidates, not just the top one. This means that they are nearly optimal when there are not significant disproportionality penalties, which happens in many real elections.

However, note that the Borda full objectives had ratios to optimal of less than 90% for 6 districts versus only 3 for STV, with 1 ratio below 15%. Thus the STV mismatches had much less severe disproportionality penalties, making them “nearly as good” as the optimal electeds, retaining a claim to significant proportionality, but benefiting from an upgrade to DTV.

Even very large data sets become tractable by our process of ballot consolidation to 100 or so slate ballots. Meanwhile, most convergences don’t require damping (section 6.1) and so are not contentious: When a convergence fails or stagnates it is a sign of an unstable or problematic configuration of clusters, symptomatic of complexity and possible controversy. A cluster set that is converged to only once out of all the initial sets is another sign of a problematic configuration, and is likewise eliminated. Another source of possible controversy arises when several possible elected sets have a full objective close to the top full objective, which often arises when the cluster disproportionality penalties are the same but the elected ratings differ. However this is like what happens currently with similar candidates whose ballot totals are nearly tied, which is the nature of the beast.

Of greater importance are how voter rankings or ratings are to be converted into points that may be compiled, especially top ranking versus bottom rankings (section 9.3). Real life experience suggests that most voters focus more on one or a few favored candidates although negative voting against especially disliked candidates will also be a factor in certain situations. Our approach in the case of ranked candidates has been to increase the points for the top ranked candidates but to reduce or eliminate the points for bottom ranked candidates, treating these as “noise” that can interfere with proportionality in typical districts. Instead we designed our rating code to explicitly include negative ratings when that is desired. However, our election data was all based on ranking, so the rating case requires more testing and development.

Where proportional representation would make the biggest difference in the US is in two situations. One is to dramatically reduce gerrymandering for congressional seats by identifying small states as single districts, with all candidates running for the same open seats, while large states would be geographically subdivided into several such districts. Of course, the politics are formidable, but so are the benefits. A second situation is to replace court-ordering gerrymandering to ensure that some districts will elect representatives for important minority populations. This could be easily applied at the county or city level, and without too much trouble for state

legislative districts. In both situations having more algorithmic tools and analysis available would certainly be helpful.

Funding None

Data All data and code are available from the author upon request. <https://github.com/Dick-Burkhart/Voting-Algorithms>

Declarations

Competing interests No potential conflict of interest or financial or non-financial interests.

References

- [1] L. Zadeh (1965), Fuzzy sets, *Information and Control*, 8:3:338–353.
- [2] P. Emerson (2011) *Defining Democracy: Voting Procedures in Decision-Making, Elections and Governance*, 2nd ed. Springer
- [3] R. Fletcher (1980), *Practical Methods of Optimization - Unconstrained Optimizatin*, Wiley & Sons
- [4] D. L. Kreher, D. R. Stinson (1999) *Combinatorial Algorithms: Generation, Enumeration, and Search*. CRC Press
- [5] C.K. Reddy, B. Vinzamuri (2014) A Survey of Partitional and Hierarchical Clustering Algorithms. In: C.C. Agarawal, C.K. Reddy (ed) *Data Clustering - Algorithms and Applications*. CRC Press, p 95
- [6] G. Kalna, D. Higham (2007) A Clustering Coefficient for Weighted Networks, with Application to Gene Expression Data. *AI Communications*, pp 263-271
- [7] J. Saram, M. Kivel, J.P. Onnela, K. Kaski, J. Kert (2007) Generalizations of the clustering coefficient to weighted complex networks. *Phys Rev E* 75:027105

- [8] E. Saari (2001), Decisions and Elections - Explaining the Unexpected. Cambridge U Press
- [9] N. Tideman (2006), Collective Decisions and Voting - The Potential for Public Choice. Ashgate, p 286